

Índice

1	Cátedra de Redes y Sistemas Distribuidos 2026	1
1.1	Objetivos	1
1.2	Protocolo HFTP	1
1.2.1	Comandos y Respuestas	1
1.2.2	Manejo de Errores	2
1.2.3	Encuadre del payload en get_slice (éxito)	3
1.2.4	Ejemplo concreto: qué responde el servidor y qué hay en el cable	3
1.3	Tarea	5
1.4	Pasos a seguir	6
1.4.1	Etapas de desarrollo incremental (orden recomendado)	7
1.4.2	Autoevaluación	8
1.4.3	Contenido del video	8
1.5	HFTP sobre Tor (obligatorio)	9
1.6	Requisitos de la entrega	9

1 Cátedra de Redes y Sistemas Distribuidos 2026

1.1 Objetivos

- Aplicar la comunicación cliente/servidor por medio de la programación de sockets, desde la perspectiva del servidor.
- Familiarizarse con un protocolo de aplicación diseñado en casa.
- Comprender, diseñar e implementar un programa servidor de archivos en Python.
- Comprender cómo el mismo protocolo HFTP puede operar a través de la red Tor (servicio oculto .onion y cliente vía SOCKS), sin modificar comandos ni mensajes del protocolo.

1.2 Protocolo HFTP

Llamaremos *Home-made File Transfer Protocol* (HFTP) a un protocolo de transferencia de archivos casero, creado por nosotros específicamente para este laboratorio.

HFTP es un protocolo de capa de aplicación que usa TCP como protocolo de transporte. TCP garantiza una entrega segura, libre de errores y en orden de todas las transacciones hechas con HFTP. Un servidor de HFTP escucha pedidos en el puerto TCP 19500.

1.2.1 Comandos y Respuestas

El cliente HFTP inicia el intercambio de mensajes mediante pedidos o **comandos** al servidor. El servidor envía una **respuesta** a cada uno antes de procesar el siguiente hasta que el cliente envía un comando de fin de conexión. En caso de que el cliente envíe varios pedidos consecutivos, el servidor HFTP los responde en el orden en que se enviaron.

- **Encoding y modelo de bytes:** El **control del protocolo** (comandos, línea de respuesta y, en get_slice **raw**, cabeceras Content-Length) es **ASCII**, con terminador de línea `\r\n`¹. En get_slice **sin** raw,

¹Ver [End of Line \(EOL\)](#): `\r` = CR, `\n` = LF, `\r\n` = CR+LF (convención habitual en protocolos de aplicación por texto, p. ej. SMTP o cabeceras HTTP).

el payload es **una sola línea** de texto **base64** seguida de `\r\n` que cierra el payload. En modo **raw**, tras las cabeceras, el cuerpo son **exactamente SIZE bytes** binarios fijados por Content-Length².

- **Comandos:** consisten en una cadena de caracteres compuesta por elementos separados por un único espacio y terminadas con fin de línea `\r\n` (CR+LF)³. El primer elemento del comando define el tipo de acción esperada y los elementos que siguen son argumentos necesarios para realizar la acción.
- **Estructura de la respuesta:** Toda respuesta del servidor tiene la misma forma conceptual que protocolos como FTP o SMTP:
 - La **primera línea es siempre la respuesta de control:** `<código> <mensaje>\r\n`, donde el código es numérico (0, 1xx, 2xx) y el mensaje es texto en ASCII.
 - **Todo lo que sigue corresponde al payload del comando** (si existe). Si el comando no devuelve datos (p. ej. un error 2xx o quit), no hay payload.
 - El **formato exacto del payload depende del comando ejecutado** y se resume a continuación (cada ítem: forma del pedido y qué responde el servidor).
 - **help** — Sin argumentos. Lista los comandos disponibles (uno por línea, fin con línea vacía `\r\n`). Facilita pruebas con telnet y depuración. Respuesta: `0 OK + \r\n + nombres de comandos + \r\n`.
 - **get_file_listing** — Sin argumentos. Obtiene la lista de archivos disponibles. Payload: nombres de archivos, uno por línea, línea vacía `\r\n` indica fin. Respuesta: `0 OK + \r\n + nombres + \r\n`.
 - **get_metadata FILENAME** — Un argumento FILENAME⁴. Payload en éxito: una línea con el tamaño en bytes (decimal ASCII) + `\r\n`⁵. Respuesta: `0 OK + \r\n + tamaño + \r\n`.
 - **get_slice FILENAME OFFSET SIZE [raw]** — OFFSET y SIZE en **bytes** del archivo, no negativos. **Éxito:** tras `0 OK\r\n`, el formato depende del modo. **Sin cuarto argumento:** el fragmento en **base64** en **una sola línea ASCII** seguida de `\r\n` que marca el fin del payload. **Con el cuarto argumento raw:** el encuadre obligatorio Content-Length (ver abajo) y **exactamente SIZE bytes** en bruto. El cliente debe implementar ambos modos.
 - **quit** — Sin argumentos. Termina la conexión. El servidor responde `0 OK` y cierra la conexión. Sin payload.

1.2.2 Manejo de Errores

El servidor responde siempre con una línea de control `<código> <mensaje>\r\n`. La semántica de los códigos es:

- **Códigos 0 (éxito):** La operación se realizó correctamente; puede haber payload según el comando.
- **Códigos 1xx (error fatal):** Tras enviar esta respuesta el servidor cierra la conexión. No se deben enviar más comandos.
- **Códigos 2xx (error recuperable):** Solo línea de control, sin payload. La conexión permanece abierta; el cliente puede enviar más comandos.

²En modo base64, el payload es ASCII terminado con `\r\n`; el cliente decodifica y obtiene los bytes del archivo. En modo raw, Content-Length fija cuántos bytes binarios leer del stream TCP.

³Ver **End of Line (EOL)**: `\r` = CR, `\n` = LF, `\r\n` = CR+LF (convención habitual en protocolos de aplicación por texto, p. ej. SMTP o cabeceras HTTP).

⁴Los nombres de archivos no deberán contener espacios.

⁵Ese número es la longitud del archivo en **bytes** en disco, no la cantidad de caracteres “visibles”: según la codificación del contenido (p. ej. UTF-8), un mismo texto puede ocupar distinta cantidad de bytes; `get_slice` también usa OFFSET y SIZE en bytes.

Códigos concretos:

- 0 La operación se realizó con éxito.
- 100 Se encontró un carácter `\n` fuera de un terminador de pedido `\r\n`.
- 101 No hay comando interpretable (p. ej. línea vacía o solo espacios hasta el `\r\n`) u otra malformación que impide identificar un comando válido⁶. No confundir con **201**, que aplica cuando el comando **sí** se reconoce pero los argumentos son incorrectos.
- 199 El servidor tuvo algún fallo interno al intentar procesar el pedido.
- 200 El comando no está en la lista de comandos aceptados.
- 201 La cantidad de argumentos no corresponde o no tienen la forma correcta.
- 202 El pedido se refiere a un archivo inexistente.
- 203 El pedido se refiere a una posición inexistente en un archivo⁷.

1.2.3 Encuadre del payload en `get_slice` (éxito)

- **Modo base64** (sin `raw`): tras `0 OK\r\n`, el servidor envía el fragmento codificado en base64 en **una sola línea** y un `\r\n` **final** que delimita el payload (sin cabecera `Content-Length`). El cliente lee hasta ese fin de línea, decodifica el base64 y obtiene **SIZE** bytes del archivo.
- **Modo raw**: tras `0 OK\r\n`, el servidor envía **obligatoriamente**:
 1. Una línea: `Content-Length: <N>\r\n`, donde `N` es decimal y `N = SIZE` (bytes del cuerpo).
 2. Una línea vacía: `\r\n` (fin de cabeceras, análogo a HTTP).
 3. Exactamente **N bytes** binarios del trozo de archivo.

Si el comando no es `get_slice` o la respuesta es un error (códigos distintos de 0), **no** aplica este payload de datos; en particular **no** se envían cabeceras `Content-Length` ni cuerpo binario `raw`.

1.2.4 Ejemplo concreto: qué responde el servidor y qué hay en el cable

Supongamos que el directorio de datos del servidor contiene **solo** el archivo `archivo.txt`. En los ejemplos siguientes, la notación `...\r\n` indica el **terminador de línea del protocolo**: los **dos bytes** CR+LF (`0x0D` seguido de `0x0A`), no la secuencia de cuatro caracteres visibles `\`, `r`, `\`, `n`.

No confundir:

- **Fin de línea HFTP**: bytes `0x0D 0x0A`. En Python, `b"\r\n"` o `"\r\n".encode("ascii")` producen exactamente esos dos bytes.
- **Cadena literal equivocada**: en Python, `r"\r"` son **dos caracteres** ASCII: barra invertida (`0x5C`) y `r` (`0x72`). Si se enviaran por el socket, **no** cerrarían un comando HFTP válido.

El **control** del protocolo (comandos, `0 OK`, cabecera `Content-Length`) es texto **ASCII** terminado en `\r\n`. El **contenido del archivo** son bytes en disco; pueden incluir `0x0D`, `0x0A` u otros valores: en modo **raw** esos bytes son **datos** y solo `Content-Length` dice cuántos bytes leer del cuerpo.

⁶Incluye el caso de una línea de pedido vacía (solo `\r\n` o espacios). **201** se reserva para comandos reconocidos con argumentos mal contados o mal formados. Malformaciones críticas (p. ej. comandos malintencionados de gran longitud) pueden provocar DoS; el implementador puede reportar error fatal.

⁷En `get_slice` cuando no se cumple `OFFSET + SIZE ≤ filesize`.

1.2.4.1 Caso A — archivo.txt con una sola línea (solo ASCII)

Contenido **exacto** del archivo en disco (UTF-8; todo es ASCII de un byte):

```
Hola, esto es una prueba!
```

Eso son **25 bytes**. El número que devuelve `get_metadata` es la cadena decimal ASCII 25 (bytes 0x32 0x35), no el byte de valor numérico 25.

En cada fila, lo indicado entre **acentos graves** es la secuencia **exacta** de bytes que el cliente envía o que el servidor responde por el socket (cada `\r\n` en esas cadenas son los dos bytes CR+LF). En `help`, el orden de nombres coincide con **COMMANDS** en `constants.py` del kickstart. En `get_slice raw`, tras `Content-Length` y el doble `\r\n` de fin de cabeceras viene el **cuerpo binario** tal cual en el cable: si todos esos bytes son ASCII imprimible (como en este ejemplo), se muestran **como caracteres** a continuación del prefijo; si hubiera bytes no imprimibles, hay que leerlos como octetos (véase el **Caso B** más abajo, con CR+LF dentro del archivo).

Pedido del cliente	Respuesta del servidor
<code>help\r\n</code>	0 OK\r\nget_file_listing\r\nget_metadata\r\nget_slice\r\n
<code>get_file_listing\r\n</code>	0 OK\r\narchivo.txt\r\n\r\n
<code>get_metadata archivo.txt\r\n</code>	0 OK\r\n25\r\n
<code>get_slice archivo.txt 0 25\r\n</code>	0 OK\r\nSG9sYSwgZXN0byBlcyB1bmEgcHJ1ZWJhIQ==\r\n
<code>get_slice archivo.txt 0 10\r\n</code>	0 OK\r\nSG9sYSwgZXN0bw==\r\n
<code>get_slice archivo.txt 0 10 raw\r\n</code>	0 OK\r\nContent-Length: 10\r\n\r\nHola, esto
<code>quit\r\n</code>	0 OK\r\n

Tras la respuesta a `quit` no hay más bytes HFTP; el servidor cierra el TCP.

En modo **base64**, cualquier byte “problemático” del archivo (p. ej. un CR real) va **dentro** de la codificación; el cliente encuentra el fin del payload leyendo hasta el **primer** `\r\n` **después** de la línea base64, no interpretando bytes dentro del archivo como fin de línea del protocolo.

1.2.4.2 Caso B — mismo nombre de archivo, pero con CR+LF dentro del contenido

Contenido en disco (el salto entre líneas son los bytes reales 0x0D 0x0A, no una metáfora):

```
Hola, esto es una prueba!  
Segunda línea.
```

(Se escribe línea sin tilde para que **todo** el archivo sea ASCII de un byte; con línea en UTF-8 habría un byte más.) En total son **41 bytes** (25 + 2 + 14): la primera línea visible, luego bytes 0x0D 0x0A, luego Segunda línea..

- `get_metadata archivo.txt` devuelve en el payload la línea 41 (caracteres ASCII '4' y '1').
- `get_slice archivo.txt 25 4 raw`: pide 4 bytes a partir del offset 25 (0-based). Esos bytes son **en orden** 0x0D, 0x0A, 0x53 (S), 0x65 (e) — es decir, CR, LF y el inicio de Segunda. El cuerpo **raw** trae

exactamente esos cuatro valores; **no** son terminadores HFTP: el cliente debe leer **4** bytes contados por Content-Length, no “hasta el siguiente CRLF” dentro del cuerpo.

- **get_slice archivo.txt 25 4 (base64)**: el servidor codifica en base64 esos mismos 4 bytes en una sola línea (DQpTZQ==); el `\r\n` que sigue a esa línea es solo el delimitador del **mensaje** HFTP.

Este contraste es el núcleo del *framing* sobre TCP: en **base64** el payload de datos es “una línea de texto” acotada por `\r\n`; en **raw** el payload binario puede contener los mismos valores byte que el terminador de línea, y por eso la longitud la fija **Content-Length**.

1.3 Tarea

Deben diseñar e implementar un **servidor de archivos en Python 3** que soporte completamente el protocolo de transferencia de archivos **HFTP**. El servidor debe ser **robusto** y tolerar comandos incorrectos, ya sea de forma intencional o maliciosa.

Evaluación: el laboratorio se considera **aprobado** o **no aprobado**; **no** se asigna nota numérica a esta entrega. Por tanto no tiene sentido hablar de «puntos extra» ni de bonificaciones sobre una calificación del trabajo práctico.

El cliente y el servidor podrán estar en máquinas distintas (misma red). El servidor puede probarse en **localhost**, en **LAN** (p. ej. una máquina del laboratorio como servidor y otra como cliente) o usando infraestructura que proporcione la cátedra; **no se exige** exponer el puerto a Internet de forma directa. **A demás**, deben **demostrar el mismo HFTP a través de la red Tor** (servicio oculto y cliente vía Tor), según **Guia_HFTP_Tor.md** y *Contenido del video*. El servidor será capaz de manejar varias conexiones a la vez.

Se recomienda que el servidor registre al menos: conexión aceptada, comando recibido, error enviado y conexión cerrada. A continuación, un ejemplo de ejecución atendiendo a un único cliente:

```
caro@victoria:~/Ayudantia/Redes$ python server.py
Serving testdata on 0.0.0.0:19500.
Connection from ('127.0.0.1', 44639)
Request: help
Request: get_file_listing
Request: get_metadata client.py
Request: get_slice client.py 0 1868
Closing connection...
```

El servidor debe aceptar en la línea de comandos las opciones:

- **-d directory** para indicarle donde están los archivos que va a publicar.
- **-p port** para indicarle en que puerto escuchar. Si se omite usará el valor por defecto.
- **-a address** (opcional) para indicar en qué dirección escuchar (por defecto `0.0.0.0`; permite usar por ejemplo `127.0.0.1` para solo localhost).

Deben utilizar el comando `telnet <dir IP> <num Port>` para enviar comandos mal formados o mal intencionados y probar la robustez del servidor.

1.4 Pasos a seguir

Los **pasos** 1–9 ordenan el trabajo del laboratorio (entorno, cronología, entregables). Los **objetivos de implementación**, las **pruebas manuales** y los **comandos** concretos de `server-test.py` están en **Etapas de desarrollo incremental** (A–G). Donde haya solapamiento, mandan las etapas: cada paso que toca código cita la etapa en la que están el detalle y las pruebas acotadas.

1. Descargar y preparar el entorno:

- Obtener el kickstarter del laboratorio desde el aula virtual.
- Descomprimirlo con: `tar -xvzf kickstart_lab2.tar.gz`
- El kickstarter incluye: estructura base del servidor a completar (**`server.py`** y **`connection.py`**), archivo de constantes (**`constants.py`**), un cliente HFTP funcional, el script de pruebas (**`server-test.py`**) —la suite completa—, scripts de autoevaluación (**`grade.py`**, **`test_metrics.py`**, **`requirements.txt`**, **`pyproject.toml`**), un **Makefile** con objetivos **`make test-tor`** / **`make test-tor-e2e`**, la carpeta **`tests/`** (**`test_tor_hftp.py`**) y **`scripts/tor/`** (coherentes con la guía), y la **Guía HFTP Tor** (**`Guia_HFTP_Tor.md`**) en el mismo directorio al descomprimir.
- *Relación con etapas:* preparación previa a cumplir el **objetivo** de la **Etapas A**.

2. Configurar un entorno virtual de Python siguiendo la documentación. Si usan `venv`, forma parte del mismo cierre de la **Etapas A**.

3. Verificar el entorno y el código base (sin servidor HFTP aún):

Comprueben que pueden ejecutar Python sobre el kickstart (por ejemplo, `python3 client.py -h`). **`server.py`** y **`connection.py`** llegan como plantilla a completar: no debe esperarse un servidor funcional ni que **`client.py`** o **`server-test.py`** pasen la suite completa hasta avanzar en las etapas de implementación. **Criterio: Objetivo y Cómo probar** de la **Etapas A**.

4. Modificar el servidor para aceptar conexiones: Seguir la **Etapas B** (aceptar conexiones, crear **`connection`**, comandos de texto y pruebas allí indicadas).

5. Implementar el resto del protocolo: Avanzar en orden por las **Etapas C, D y E** (`get_slice base64`, `get_slice raw`, errores y robustez). En cada etapa están los objetivos, **`telnet`** / **`client.py`** cuando corresponda y los **`unittest`** acotados.

6. Validar con el cliente (**`client.py`**): Como indica el apartado **Cómo probar manual** (u homólogo) en cada etapa ya alcanzada; por ejemplo **Etapas C** para descargas en modo por defecto.

7. Pruebas automatizadas: Después de cada etapa, ejecutar **solo** el bloque de tests listado en esa etapa (servidor en marcha en otra terminal, salvo que usen **`grade.py`**). La invocación **sin filtros** —`python3 server-test.py`— que corre **toda** la suite funcional es la de la **Etapas G**. Para comprobar **de una vez** tests, cobertura sobre `connection.py`/`server.py`, complejidad ciclomática y ruff, usen **`python3 grade.py`** (mismo criterio que la corrección automática; véase **Autoevaluación**).

8. Varios clientes simultáneos (hilos): **Etapas F**.

9. HFTP sobre Tor (obligatorio): El objetivo figura en la **Etapas G**; el detalle operativo está en **`Guia_HFTP_Tor.md`** (incluida en el tarball del kickstart, misma carpeta que `server.py`). Mismo HFTP por TCP, servidor detrás de servicio oculto (`.onion`) y cliente vía red Tor, sin cambiar comandos ni mensajes del protocolo. No hace falta reproducir aquí instalación de Tor ni `torrc`;

conviene entender *onion service* y proxy SOCKS ([Tor Project](#) — documentación general, no sustituye la guía del lab).

1.4.1 Etapas de desarrollo incremental (orden recomendado)

Esta lista es la **referencia** de hitos implementables y de comandos de prueba. Los [pasos a seguir](#) anteriores ordenan el curso y remiten aquí para no duplicar objetivos ni unittest. No son entregas obligatorias a la cátedra salvo lo que indique el aula virtual: sirven para organizar el trabajo y saber cuándo “cerrar” un bloque.

Convención para las pruebas automáticas: en una terminal dejen el servidor corriendo, por ejemplo `python3 server.py -d testdata -p 19500`. En otra, desde el directorio donde está `server-test.py`, ejecuten los comandos indicados (usan el módulo `unittest` de la biblioteca estándar).

- **EtapA A — Cimientos del proyecto**
 - **Objetivo:** proyecto descomprimido, entorno virtual si lo usan, archivos del kickstart accesibles.
 - **Cómo probar:** `python3 -c "import constants, client"` sin error; `python3 client.py -h` muestra la ayuda del cliente (puede fallar la conexión si aún no hay servidor: es esperable).
- **EtapA B — Conexión TCP y comandos “solo texto”**
 - **Objetivo:** `server.py` acepta conexiones y crea `Connection`; implementar al menos `quit`, `help`, `get_file_listing` y `get_metadata` con el formato del protocolo (sin `get_slice` o con una respuesta provisional clara).
 - **Cómo probar manual:** `telnet` a la IP y puerto del servidor: `help`, `get_file_listing`, `get_metadata` sobre un archivo que exista en el directorio de datos, `quit`.
 - **Cómo probar automático (servidor en marcha):**
`python3 server-test.py TestHFTPSTestServer.test_connect_and_quit TestHFTPSTestServer.test_help TestHFTPSTestServer.test_quit_answers_ok TestHFTPSTestServer.test_lookup TestHFTPSTestServer.test_empty_file_listing TestHFTPSTestServer.test_get_metadata TestHFTPSTestServer.test_get_metadata_empty TestHFTPSTestServer.test_context_manager`
- **EtapA C — `get_slice` sin `raw` (base64 y fin de línea)**
 - **Objetivo:** comando `get_slice` `FILENAME` `OFFSET` `SIZE` **sin** cuarto argumento: tras `0` `OK`, payload en **base64** terminado en `\r\n` (sin cabecera `Content-Length`).
 - **Cómo probar manual:** `telnet` o `client.py` descargando un archivo pequeño (el cliente usa este modo por defecto).
 - **Cómo probar automático:**
`python3 server-test.py TestHFTPSTestServer.test_get_full_slice TestHFTPSTestServer.test_partial_slices TestHFTPSTestServer.test_retrieve_full_file`
- **EtapA D — `get_slice` con `raw` y `Content-Length`**
 - **Objetivo:** cuarto argumento `raw`; tras `0` `OK`, cabecera `Content-Length`, línea vacía y **exactamente** `SIZE` bytes binarios; rechazar variantes incorrectas (p. ej. `rawx`).
 - **Cómo probar automático:**
`python3 server-test.py TestHFTPSTestServer.test_get_slice_raw TestHFTPSTestServer.test_get_slice_raw_partial TestHFTPSTestServer.test_get_slice_rejects_bad_raw_token`
- **EtapA E — Errores y robustez (un cliente a la vez)**
 - **Objetivo:** códigos y casos límite; el servidor puede seguir atendiendo **una** conexión completa por vez.

- **Cómo probar automático:**
python3 server-test.py TestHFTPErrors TestHFTPHard
(incluye EOL incorrecto, argumentos, archivos inexistentes, offsets, permisos de archivo, entrada fragmentada, listados largos, etc.)
- **Etapas F — Varios clientes (hilos)**
 - **Objetivo:** varias conexiones simultáneas atendidas correctamente.
 - **Cómo probar automático:**
python3 server-test.py TestHFTPMultiClient
- **Etapas G — Cierre**
 - **Objetivo:** revisar el repositorio, cumplir *Requisitos de la entrega y Contenido del video*; **demonstrar HFTP extremo a extremo vía Tor** (servicio oculto y cliente a través de la red Tor), según `Guia_HFTP_Tor.md`, sin alterar el protocolo.
 - **Cómo probar:** con el servidor en marcha: python3 server-test.py sin filtros (toda la suite funcional). Para el cierre completo alineado con la corrección (incluidas métricas), usar **python3 grade.py** tras `pip install -r requirements.txt` (véase [Autoevaluación](#)).

1.4.2 Autoevaluación

Tras `pip install -r requirements.txt` en el directorio del kickstart:

```
$ pytest -v
```

Deben pasar **todos** los tests, incluidos los de `test_metrics.py` (complejidad ciclomática y ruff sobre `connection.py` y `server.py`). Los tests de `server-test.py` requieren servidor HFTP en marcha **salvo** que ejecuten la suite a través de `grade.py`, que levanta el servidor bajo **coverage**, corre `pytest` y comprueba la **cobertura mínima** sobre los módulos del servidor (umbrales en `pyproject.toml` / `grade.py`).

Resumen único (recomendado):

```
$ python3 grade.py
```

Muestra si cumplen tests, cobertura, complejidad y ruff. En el repositorio de la cátedra pueden usar **make grade** (equivale a instalar dependencias y ejecutar `grade.py` sobre **solucion/**).

1.4.3 Contenido del video

El video es el único entregable audiovisual junto con el código en el repositorio: **no** se pide informe escrito ni presentación en diapositivas. Lo que sigue no es un cuestionario ni un guion rígido: son **temas y disparadores** para que elijan qué mostrar y qué contar. Deben integrarlos en un relato **cohesivo** (por ejemplo: breve demo, recorrido por el código o el protocolo, y reflexión), evitando el formato “pregunta 1, respuesta 1; pregunta 2, respuesta 2”.

Temas obligatorios (deben quedar claros en algún momento del video, en el orden que prefieran):

- **Varios clientes a la vez:** estrategias posibles para atender *múltiples clientes simultáneamente* y qué implicaría en el diseño de su servidor (aunque su entrega use una opción concreta, pueden contrastar con alternativas).
- **Servidor y cliente en distintas máquinas:** una demostración o relato breve probando el servidor en una máquina del laboratorio y el cliente en otra, hacia la IP del servidor; y la diferencia práctica entre escuchar en `localhost`, `127.0.0.1` y `0.0.0.0`.

- **HFTP sobre Tor:** demostración o relato que muestre el servidor publicado como servicio oculto (dirección `.onion`) y el cliente accediendo a través de la red Tor (p. ej. con `torsocks`), con **listado o transferencia verificable**; pueden apoyarse en **Guia_HFTP_Tor.md**.

Profundización adicional (elijan **al menos dos** ejes de la lista; pueden mezclar apartados y enlazarlos con lo que ya mostraron):

Protocolo y datos

- Modo base64 y modo raw en `get_slice`, y qué ilustran sobre *framing* sobre un stream TCP.
- Por qué en base64 basta el `\r\n` final y en raw hace falta Content-Length (bytes no interpretables como texto).
- Tamaño en **bytes** frente a “cantidad de caracteres”; ejemplo con UTF-8 u otra codificación.

Servidor y red

- Comportamiento ante un cierre de TCP **sin** quit; cómo lo refleja su implementación.
- Por qué TCP encaja mejor que UDP para este tipo de servicio.

Concurrencia y diseño

- Recursos compartidos entre hilos (p. ej. directorio de datos) y precauciones al leer o listar.
- Contraste breve entre modelo con **hilos** y multiplexación con **poll** o **select**.

Errores y robustez

- Significado de los códigos **100**, **101** y **199**, con un ejemplo de situación que hayan podido reproducir (o los que correspondan).
- Cómo evitan que entradas incorrectas o comandos muy largos dejen el servidor colgado o sin límites de memoria.

Seguridad (mínima)

- Riesgos obvios al publicar un directorio (p. ej. rutas tipo `../`) y qué hicieron para acotarlos, aunque sea de forma mínima.

1.5 HFTP sobre Tor (obligatorio)

Deben **demostrar HFTP sobre Tor**: servidor accesible vía dirección `.onion` y cliente operando a través de la red Tor (p. ej. con `torsocks`), con transferencia o listado verificable, como cierre integrador (capas, TCP, anonimato y servicios ocultos). El protocolo HFTP **no** se modifica. El detalle operativo está en **Guia_HFTP_Tor.md**, el mismo archivo que reciben **dentro del tarball** del kickstart. Para contexto sobre qué es Tor y buenas prácticas, sirve la documentación oficial del [Tor Project](#) (complementaria, no reemplaza la guía del lab). Esta parte forma parte de los **requisitos de aprobación** del laboratorio (incluida la demostración en el video; véase *Contenido del video*).

1.6 Requisitos de la entrega

- El laboratorio se evalúa como **aprobado** o **no aprobado**; **no** lleva nota numérica ni bonificaciones por puntos.
- Las entregas serán a través del repositorio Git provisto por la Facultad para la Cátedra, con **fecha límite indicada en el cronograma del aula virtual**.

- **Asegúrense de usar un `.gitignore` adecuado:** no deben subir al repositorio archivos generados, datos de prueba (p. ej. directorios `testdata`), entornos virtuales, archivos temporales ni credenciales. El kickstarter puede incluir un `.gitignore` de referencia; comprueben que cubra todo lo que no deba versionarse.
- Junto con el código deberán entregar **un video de 10 ±1 minutos** (como en laboratorios anteriores de la materia). Debe cumplir lo indicado en la sección *Contenido del video*: **los tres temas obligatorios** y **al menos dos ejes** de la lista de profundización, integrados de forma natural en el relato (no como lista aislada de respuestas). Estructura sugerida (pueden adaptarla):
 1. **Introducción al proyecto:** contexto, objetivos, importancia de un protocolo de transferencia de datos.
 2. **Conceptos:** paradigma cliente/servidor y sockets; diferencia entre Stream (TCP) y Datagram (UDP); qué es FTP y base64; en `get_slice`, **contraste entre delimitar el payload con fin de línea** (modo base64: una línea base64 + `\r\n`, sin `Content-Length`) y **contar bytes con `Content-Length`** (modo raw: cabecera + cuerpo binario de longitud fija); qué pasa si un archivo contiene `\r\n` y cómo lo resuelve su código.
 3. **Desarrollo del servidor** (comandos, manejo de conexión, fragmentos de código o demo en vivo).
 4. **Red y concurrencia** (encajar aquí, si encaja, los temas obligatorios: multicliente, prueba entre máquinas, `127.0.0.1` vs `0.0.0.0`, demostración vía Tor, etc.).
 5. **Profundización:** al menos dos ejes elegidos de la lista, enlazados con lo ya mostrado.
 6. **Errores y dificultades enfrentadas**
 7. **Conclusiones** (incluir relación con el lab anterior de APIs)
- Código con estilo PEP8. Trabajo grupal; no está permitido compartir código entre grupos.
- **Condiciones técnicas de aprobación (resumen):** suite que exige `grade.py` (o equivalente): tests funcionales vía `server-test.py`, cobertura mínima sobre `connection.py` y `server.py`, límite de complejidad ciclomática por función y `ruff` sin incidencias en esos archivos; además, **demostración de HFTP vía Tor** según `Guia_HFTP_Tor.md` y lo pedido en *Contenido del video*. Detalle en [Auto-evaluación](#).